

going to your 1Password app and then to Preferences > Watchtower.

1. Use the "Security Audit"

functionality of 1Password to meet the password standard.

It will report reused passwords, weak passwords, accounts that are missing 2-factor authorization, and so forth that can then be fixed.

1. Do not copy passwords from inside a 1Password vault to a personal password vault or other password store. 1Password should be the only password vault used for teams. Team passwords should not be duplicated or placed in personal password vaults where they can potentially be exposed to compromise.

1. When asked security questions (what is your favorite pet, etc.) do not answer truthfully since that is easy to research. Make up an answer and write both the question and answer in 1Password. Consider using the Password Generator function in 1Password for this.

1. During offboarding, your 1Password account is deleted, which includes the Employee vault in the GitLab team account. If you want to keep your personal passwords, please copy/move them to your Primary vault which you will have if you signed up for an individual account before joining the GitLab Team account.

1. Deprecated When documenting the location of shared credentials in the handbook refer to the items with NAMEOFSITE credentials in VAULTNAME. For example:

"for access please see the AOL credentials in the Luddite vault".

- Deprecation note: This is for existing accounts only. New accounts should be created by creating an issue to add it to Okta.

1Password for teams

1Password for Teams stores all Vaults on the 1Password servers and allows for sharing between multiple people on the same team. Every GitLab team member who needs access to a shared vault should consult their departments for any shared vault information.

Each member of the team has a vault called Employee which only you can see, and allows you to store personal credentials within the GitLab team's account.

To really get the full benefit of 1Password, you'll need to hook our Business account up to one of the native apps.

Adding the GitLab team to a 1Password app

This guide will cover setting up the macOS app. It's their lead platform and is the most up-to-date. These instructions may or may not work for the Windows version. If you use 1Password 6 without a 1Password.com account, make note of this.

1. Download and install the 1Password macOS app.

1. Launch the app.

1. Click "Sign in to your 1Password account" button. If there is no such button please follow the instructions for updating 1Password.

Now you'll need the Emergency Kit PDF that 1Password told you to save when

you registered your Business account. Note: Store the Emergency Kit safely. Store a copy of the Emergency Kit on a USB flash drive or print a copy and store it in a vault at home or safe deposit box · somewhere not online or accessible by anyone other than yourself.

If you saved it as a digital PDF file:

1. Open the PDF file
1. Click Scan QR Code
1. Open the PDF file with the scanner by clicking on the camera icon

```
<div style="text-align:center;">  
    
</div>  
<br>
```

If you printed the PDF:

1. Click Sign In Manually
1. For Team URL enter gitlab.1password.com
1. For Account Key enter the Account Key from your Emergency Kit
1. For Email Address enter your @gitlab.com email
1. For Master Password enter the password to your Teams account (not the password you created above when you chose "I'm a new user")

After the Team is added, you should see some notifications about vaults being added to 1Password. By default you'll have the Private vault, but may have access to others.

Shared Vault Etiquette

1. In general, 1Password items should be self-documenting. This is especially important for items in team vaults such as those to which a team member gains access once part of the GitLab team or they are granted access via their baseline entitlements issue. Always make sure the title, username, and notes fields are meaningful.
1. Deleting items is fine as they are kept in the "recently deleted" tab indefinitely. It's also fine to leave them in their respective vault but delete the account on the system associated with the item. In the second case, and in order to prevent confusing team members, edit the title (e.g. [Deleted on 2024-02-14]) to reflect the purposeful deletion of the account and explain briefly who and why removed it in the notes.

Vaults

Click the Vault Selector in the upper-left corner of the window:

```
<div style="text-align:center;">  
    
</div>  
<br>
```

GitLab team members have access to a Employee vault by default, which is your hosted, private vault that is part of the GitLab 1Password for Teams account. Since the Employee vault is part of the GitLab Business account, it should be thought of as company property (like the @gitlab.com email account), however the vault cannot be viewed by anyone else on the team, including admins. If you choose to store truly personal information in the Employee vault, it opens up the possibility that you would be separated from this information if you offboard. Such truly personal information is therefore better to store in your Primary vault, which is associated with you instead of with the GitLab Teams account, assuming that you added an individual account.

People may request access to other vaults such as shared vaults that their teams/departments have created.

Browser extension

Go to Browser extensions and install the extension for whatever browser you're using. You should not need a beta version here.

With the extension installed, you should be able to go to a site that you have credentials stored for in 1Password and log in:

```
<div style="text-align:center;">  
    
</div>
```

If you don't see the site listed in the results window, make sure you're using the correct vault:

```
<div style="text-align:center;">  
    
</div>
```

Saving logins

When 1Password detects a login form submission, it may ask if you want to save the login with a dialog like this:

```
<div style="text-align:center;">  
    
</div>
```

If you do want to save it, make sure the appropriate Vault is selected first.

Managing SSH keys

Starting with version 8, 1Password can operate as the single source of truth for your SSH keys.

This includes generating private keys, storing them securely, filling your public keys in to sites like GitLab.com, and unlocking the keys automatically when performing git operations.

More information is available in the official documentation.

CLI integration

During development it is often necessary to use test secrets locally on your development machine.

Even though these secrets have a limited scope of access, it's still best practice to avoid writing them unencrypted to disk.

Manually loading them into environment variables or using them for CLI commands directly also risks them being written to disk as part of your shell history.

For a more secure approach, use 1Password's CLI utility which can help manage local secrets securely.

1Password CLI integration supports secure handling of secrets used in command line tools, config files, and scripts executed on your laptop.

To setup the CLI integration, follow the getting started guide.

It is recommended to store secrets such as personal access tokens in 1Password. Avoid storing secrets in unencrypted files or using them directly on the CLI. Many shells write a command history to disk and where the secret is used literally on the command line this will be written to the command history unencrypted.

Example for configuring glab with 1Password CLI (requires 1password version 8 or higher):

- Store your access token in 1Password. In the entry for your GitLab account, create a new section pat and add a field api. Insert the value of your PAT into the newly created field api.

```
<div style="text-align:center;">  
    
</div>
```

- Store a secret reference to the access token in an .env file.

The environment variable name goes before the equals sign, and the path to the secret in 1Password goes after.

You can store multiple variable secret pairs per file when you need to use secrets together.

The paths can be obtained by viewing the credential in the 1Password GUI, clicking the dropdown menu and selecting "Copy Secret Reference".

sh

format is op://vault-name/item-name/[section-name/]field-name

```
echo "GITLABTOKEN=op://Private/GitLab/pat/api" > $HOME/.gitlab-pat.env
```

- Run processes using the secrets

When running processes which need the secrets, run it under the control of 1Password, so it can pass the variables correctly.

To do this, prepend `op run --env-file="<path to your env file>" --` to the command you wish to run, and replace any secrets with the relevant variable

So for example

```
shell
glab api version
```

would become:

```
shell
op run --env-file=$HOME/.gitlab-pat.env -- glab api version
```

Where the process is interactive or has dynamic output (progress bars, etc) you may need to disable masking to make the display work correctly. You can do this by adding `--no-masking`

Manually adding `op run` every time for common tasks is cumbersome, so check out the tools and tips section for more user-friendly configurations.

Several accounts and unlocking the app

If you are planning to use both the GitLab team account and a separate individual account you should first add your separate individual account to the app first (Preferences > Accounts). By doing this you will be able to unlock the 1Password app using the Master Password of the individual account.

If you were using 1Password before joining GitLab, and you receive a prompt titled Migrate To Account, choose I'll move later. There is no harm in doing this, and it is easy to move items between vaults.

1Password for your private passwords{1password-private-use}

You are encouraged to use 1Password for your private passwords, not related to your work at GitLab.

This makes it less likely for a security breach to occur. You can purchase a standalone license or start an individual subscription, or take advantage of the complimentary 1Password for Families feature, which you can share with up to 5 family members.

Two factor authentication and time-based one time passwords

As stated in the GitLab Password Standards, the usage of 2FA is mandatory for all GitLab team members. Users without 2FA enabled that are stale for over 30 days will be blocked/suspended

until resolved. This improves the security posture for both the user and GitLab. If any systems provide an option to use SMS text as a second factor, this is highly discouraged. Phone company security can be easily subverted by attackers allowing them to take over a phone account.

Okta is configured such that it only supports the use of WebAuthn. 1Password TOTP should only be used when WebAuthn is unavailable.

A FIDO2/WebAuthn hardware token can be used as a secure and convenient 2-factor authentication method for Okta, Google Workspace, GitLab instances, and many other sites. If you do not have one, you may request one via the Yubibot. GitLab's standard is Yubico's YubiKey. For more information on FIDO2/WebAuthn, visit the Tools and Tips page.

If shared access to a single account is required by multiple team members, for example, a social media account, an Okta new application setup Issue should be created. The credentials will be stored and shared via Okta.

If you find an existing shared account in 1Password, create an Issue to get it migrated to Okta.

1Password provides an alternative solution that does not require using your smartphone: 1Password Time-based One Time Passwords (TOTP). 2FA codes are displayed directly in the 1Password app running on your laptop (Note: this can not be set up via 1Password browser extension or 1Password web app).

To enable TOTP for a saved account:

1. Open 1Password app
1. Go to the item for which you want to set up TOTP
1. Click Edit in the bottom right corner
1. Add a new field by clicking on the field type dropdown (it offers a "Text field" by default)
1. Select One-Time Password

```
<div style="text-align:center;">  
    
</div>
```

1. Click QR code icon that appeared

```
<div style="text-align:center;">  
    
</div>
```

1. Scan QR code using the transparent window
1. Click Save
1. 2FA code should be displayed now

Please refer to demo video 1password TOTP setup

Please refer to the 1Password blog for more information on how TOTP works.

If scanning the QR code using the "transparent window" with the 1Password Mac app fails on a recent macOS, please consider using the 1Password iOS app instead. This mechanism works the same way, and supports Touch ID to login.

If unsure which mechanism to use, we require using WebAuthn (when possible) as a TOTP for 2FA.

Follow this guideline when getting a new mobile device, if you are using Google Authenticator as a TOTP mechanism.

There may be cases where TOTP might be used with a non-GitLab account. If you have any questions and need to speak with the Security Team, you can contact Security

Passkeys

1Password has implemented a feature supporting for Passkeys, which are the credentials used in FIDO2/WebAuthn on sites such as Okta. This feature can allow passwordless sign in to sites that require username and password. Passkeys can also serve as a type of MFA. Passkeys store a public key in the app and a private key within the 1Password vault.

Setting up Passkey is simple. Open a login page or select the sign in button on the website or app. You'll typically see a prompt to complete an authentication step, usually with Biometrics, or YubiKey. You may see something like the image below.

```
<div style="text-align:center;">
  
</div>
<br>
```

At that point, the key pairing is created and 1Password will use this method of sign in going forward. Note, not all sites are supported at this time. If you're curious about setting this up on a site you use, you can always open 1Password, select that login tab, and see if the "Passkey Available" notification is showing.

```
<div style="text-align:center;">
  
</div>
<br>
```

You may need to follow different steps with the specific sites in order to complete Passkey setup, follow the prompts of the site and 1Password and you'll be set for logging in with a Passkey via 1Password.

Example usage {1password-example-usage}

This is an example of how [Robert](https://gitlab.com/rspeicher), one of our developers, uses 1Password:

- > Once you fully commit to using 1Password to manage all of your security
- > information, it really does make life easier.

- >
- > I memorize one strong password and let the app generate
- > everything else. Every site I use has a unique password that I can't
- > compromise because I don't even know it, and a hacked site can't compromise
- > it because the password is never re-used on another site.
- >
- > I store my shipping and credit card info in 1Password and use the browser
- > extension to quickly fill out shipping and billing information on shopping
- > sites.
- >
- > I store my passport data, along with a digital scan, in 1Password; driver's
- > license info and scan; insurance info; software license keys; any important
- > information that needs to be secure but still easily accessible when I need it,
- > from anywhere. I sync my personal vault to my personal iCloud so it's
- > available on my phone, tablet, laptop, and desktop.
- >
- > Even my 1Password for Teams account information is stored in my personal
- > Primary vault, with the Emergency Kit PDF as a secure attachment.
- > I have no idea what the password is. I've never actually typed it. And that's
- > the idea:

```
<div style="text-align:center;">
  
</div>
```

Traveling with 1Password{travel-mode}

When traveling with a device that has access to the GitLab 1Password vaults, be sure to enable Travel Mode in 1Password. Travel Mode removes copies of any 1Password vaults that are not tagged as "safe for travel" from your mobile devices. None of the GitLab shared vaults are marked as safe for travel so you will need to either create a dedicated travel vault or mark your Private vault as safe for travel.

Once you have enabled Travel Mode open 1Password on each device you will be taking with you so that it can sync with 1Password.com and remove any vaults that cannot be used while traveling.

For more information on Travel Mode and how it works, see the AgileBits blog.

Securing Docker Registry User Credentials

Docker can store user credentials in an external credential store as a more secure alternative to storing credentials in the Docker configuration file.

Using osxkeychain (macOS)

To configure Docker to use osxkeychain for secure credential storage, follow these steps:

1. Install the docker-credential-osxkeychain helper via Homebrew:

```
sh
brew install docker-credential-helper
```

1. Configure `~/.docker/config.json` to use `osxkeychain` as your Docker credstore:

```
json
{
  "credsStore": "osxkeychain"
}
```

1. Log in using `docker login registry.gitlab.com`, and enter your email and password (PAT) when prompted.
1. Validate that the credentials were not saved as base64-encoded text in `~/.docker/config.json`.

Using pass (Linux)

Prerequisites

1. Create a GPG key
1. Install pass

To configure Docker to use pass for secure credential storage, follow these steps:

1. Download the latest `docker-credential-pass` binary from the releases page.
1. Make the `docker-credential-pass` binary executable with `chmod +x docker-credential-pass`.
1. Move the `docker-credential-pass` binary to your `$PATH` (e.g., `sudo mv docker-credential-pass-v0.8.0.linux-amd64 /usr/local/bin/docker-credential-pass`).
1. Obtain and copy the GPG key ID that pass will use for encryption via `gpg --list-secret-keys --keyid-format LONG`.
1. Initialize pass with `pass init <gpg-key-id>`.
1. Configure `~/.docker/config.json` to use pass as a credstore:

```
json
{
  "credsStore": "pass"
}
```

1. Log in using `docker login registry.gitlab.com`, and enter your email and password (a valid token) when prompted.
1. Validate that the credentials were not saved as base64-encoded text in `~/.docker/config.json`.

SECTION: security/performance-indicators.md

{{% alert title="Warning" color="warning" %}}

This page is deprecated and has been moved to the internal handbook.

See <https://gitlab.com/gitlab-com/gitlab-security/security-department-meta/-/issues/1685> (internal issue).

{{% /alert %}}

SECTION: security/pgp_process.md

Install GPG Keychain and import PGP Keypair

On a Mac, download and install the GPG Keychain application. Download the keypair file from the Support vault. It's attached to the 'security@gitlab.com PGP Keypair' item. Open the GPG Keychain application and import the keypair file. It will ask for a password. Use the password saved on the vault item.

Now you will be able to encrypt, decrypt, and share the public key with others.

Share the Public Key

Right-click on the key in the GPG Keychain application and click 'Export'. Ensure the dialog box does not have 'Include secret key in exported file' checked. Save the exported file and send it to the requestor.

Researchers that wish to report a possible security issue should refer to our Responsible Disclosure Policy. Disclosure via email will receive an automated response configured in Zendesk, and be automatically closed.

Decrypt Text and Files

If the encrypted data comes to the security email as text, decrypting is very easy. Copy the entire block of encrypted text, right-click, go to 'Services', then 'OpenPGP: Decrypt Selection to New Window'. After a few seconds a new window will appear with the decrypted text.

If the encrypted data comes as an attachment, start by downloading the file to your workstation. Navigate to the file in your Finder. Right-click the file, go to 'Services', and choose 'OpenPGP: Decrypt File'. The decrypted file will be placed in the same directory as the encrypted file. It may have a different name, though.

SECTION: security/responding-to-ransomware.md

Ransomware is a persistent threat to many organizations, including GitLab. In the event of a

ransomware attack involving GitLab assets, it's important to know the existing response procedures in place. Given the variability of targets in such attacks, it's critical to adapt to existing circumstances and understand that disaster recovery processes are in place to avoid paying any ransom. GitLab's red team has done extensive research to determine the most likely targets to be affected. As a result, the following guidelines are intended to help bootstrap an efficient response to protect the organization.

Critical First Steps:

- Engage the SIRT team as soon as a ransomware attack is detected
- The SIRT team will then follow the incident response guide and incident communication plan and reference the relevant run book.
- Responders should leverage GitLab's established rapid engineering response plan during the mitigation phase.
- The Business Continuity & Disaster Recovery Controls handbook page should be referenced for relevant information.

Relevant Teams:

Depending on the impacted resources, the following teams should be engaged and made aware of the issue created for the rapid engineering response. Note that this is not a comprehensive list depending on impacted assets.

- Database: Disaster Recovery Team - responsible for disaster recovery strategy for the PostgreSQL database.
- Infrastructure Team - availability, reliability, performance, and scalability of GitLab SaaS software
- Infrastructure Security Team - infrastructure teams stable counterpart focused on cloud infrastructure security, best practices, and vulnerability management
- Business Technology Engineering - endpoint and systems access management
- Support Team - responding to customer or employee inquiries regarding system outages
- Legal & Corporate Affairs
- Security Assurance - assuring the security of GitLab as an enterprise application
- Marketing - accurately represent GitLab and our products in our marketing, advertising, and sales materials.

Communications:

Once we've determined that we need to communicate externally about an incident, the SIMOC should kick off our Security incident communications plan and key stakeholders will be engaged for collaboration, review and approval on any external-facing communications. Note: if customer data is exposed, external communications may be required by law.

SECTION: security/root-cause-analysis.md

What is an RCA?

Root Cause Analysis (RCA) is a process to ultimately identify the root problem of an issue so

that we may prevent it from occurring again. You can learn more about RCAs [here](#).

To do this a DRI and all relevant other stakeholders from Security and the affected teams (as determined by the DRI) systematically work through a set of questions and discussion topics, as defined in our RCA Template.

What is an RCA vs a symptom?

It's important to distinguish between root cause and symptoms of vulnerabilities. For example, malware compromise of a system is a symptom of a root problem that must be resolved to keep it from getting compromised again. The root cause could be that there is a golden image that needs to be patched, in order to prevent the problem from being created perpetually. Follow up action items may be threefold: 1) Patch the golden image 2) Patch all vulnerable systems 3) Ensure that the patching procedures address gaps in patching cycles.

Why do a RCA for S1 vulnerabilities?

It's important to learn from our past mistakes in order to prevent the same or similar ~"severity::1" issues from repeating in the future. The expectation is that we can both identify and address the root problem as well as discover other similar attack vectors related to the root cause.

Initiating an RCA

After a ~"severity::1" security issue has been resolved, an RCA should be initiated by opening an issue using the RCA template in the RCAs project.

Due date is automatically set to 30 days.

A specific DRI must be assigned to the issue. By default, this is the security engineer assigned to the incident who has the most technical context. The DRI can be someone else as long as the assignee and a SIRT manager agree. The DRI should be someone capable of understanding the technical context. The DRI role is mainly administrative, with responsibilities described below.

The DRI is also responsible for setting the correct metadata labels on the incident issue.

Progressing an RCA

The DRI is responsible for reminding others to contribute to the RCA, and for ensuring any stakeholders from relevant Product Groups or other parts of the company, which have been involved in the incident, have been given an opportunity to contribute.

The DRI is responsible for creating issues for corrective actions the team have identified and assigning a DRI for those follow up issues. The DRI is empowered to make decisions about which corrective actions are specific, measurable, achievable and relevant enough to create an issue on. Because not all issues raised by team members may meet this criteria, the DRI is also empowered to decline corrective actions while at the same time explaining why it was declined. Leadership will assist with resolving any potential conflicts.

Note the DRI does not need to come up with all the ideas! This is mainly an administrative

task.

The DRI for an RCA should aim to meet these timeframes:

- T+2 weeks: involved people have been given an opportunity to contribute.
- T+4 weeks:
 - All identified corrective actions have issues created and DRIs assigned
 - The RCA issue description is in a tidy & readable state

When is an RCA considered complete?

The RCA is considered complete when the tasks in the RCA issue are marked as completed and the issue is closed. This means that the root cause of the vulnerability is well understood and we have a path forward to reduce the likelihood of a similar vulnerability happening again. For example this can be a custom SAST rule, new security enhancement addressing the vulnerability class holistically, secure coding training, threat model, more secure application settings, etc.).

Issues for corrective actions can be labeled with ~"corrective action" and an SLO will apply depending on the severity.

Where can I find past RCAs?

In the rcas project closed issues.

How can I improve our RCA process?

Open an MR to update this page, and update the RCA Template.

RCA metadata

To prepare and aid the RCA process a set of metadata should be applied by the DRI to the respective incident issues by using GitLab labels before the actual RCA process is kicked off. This metadata is based on a set of pre-defined scoped labels:

- ~Incident::Category::
- ~Incident::Classification::
- ~Incident::Organization::
- ~Incident::Origin::
- ~Incident::Source::

When the decision is made for a particular incident that an RCA is needed the ~RCA::todo label should be applied to the incident issue by the DRI.

For ~severity::1 issues an RCA is always needed, for other issues of lower severity the need for an RCA is decided on a case-by-case basis.

Along with the ~RCA::todo a ~Label Review::needed label is applied to the issue. The DRI for the RCA now should review the scoped Incident:: labels for accuracy and adjust those where needed. Once all labels are

reviewed and adjusted the
~Label Review::done label should be applied by the DRI.

Once the labels are reviewed and accurately set an RCA issue can be created.
When the RCA issue is closed the ~RCA::todo label can be swapped for the ~RCA::done label.

Category labels

Having a not too broad and not too specific list of ~Incident::Category:: labels for our incidents we're able to gather meaningful data on our problem areas. Subsequent root cause analysis can show how to tackle the individual issues, however it should be generalized with the aim to mitigate whole categories of issues effectively. This might be done with joint RCAs in cases where we have multiple issues in the same category in a rather short time frame.

SECTION: security/secure-coding-training.md

This page contains information on secure training initiatives sponsored by the GitLab Security team.

Security Development Process

For information on developing security fixes in GitLab, please see the Patch Release runbook for preparing security fixes. (Required)

Secure Coding Guidelines

The GitLab Secure Coding Guidelines (Required) cover how to address specific classes of vulnerabilities that have been identified in GitLab.

Secure Code Warrior

GitLab uses Secure Code Warrior to provide ongoing secure coding training. Eligible team members can log in via Okta.

If you think you should have access but don't see it in Okta, check this list of roles below.

Using Secure Code Warrior

Team members may be assigned Courses. You can complete a Course in any of the available languages, for example Ruby, Go, Vue.JS, Python, or others. Follow these instructions to find and complete your course: <<https://help.securecodewarrior.com/hc/en-us/articles/360040802312-Courses-Module-Overview>>. Visit <<https://portal.securecodewarrior.com//courses/course-list>> to see your Course list.

We recommend self-selecting Training in the form of Challenges and Missions every month to learn about secure coding and to keep your skills sharp. These hands on examples will help you write better code and spot potential security concerns during code reviews. Learn more at

<<https://help.securecodewarrior.com/hc/en-us/articles/360035983992-Training-Module-Overview>>.

From time to time the Security Department might also run Tournaments. More information on these will be released ahead of any tournament start date.

Getting help

Visit the security-secure-code-warrior Slack channel to chat, ask questions, give feedback, or get help.

Found a bug, confusing content, or incorrect answer? Please use Secure Code Warrior's built in bug reporting by selecting "Help" in the top right, then selecting "Submit bug or feedback". This will help improve the platform for all users.

Mandatory training

GitLab requires that team members with development responsibilities complete a specific secure code training course at least annually. Each team member must complete it by the end of the Financial Year at the latest.

The training is provided via Secure Code Warrior and the course is titled "GitLab Secure Development Short Course". It takes 1-3 hours. The requirement is only to complete the course: hints may be used, and incorrect answers are OK. Have fun and learn!

<details>

<summary markdown="span">Expand to see the roles who must complete mandatory training.</summary>

The criteria used to determine inclusion were:

- Does the role description include writing or reviewing code that may interact with GitLab RED data, and in a language supported by Secure Code Warrior?
 - This is typically Individual Contributors and their direct Managers. Senior Managers and above are excluded, at present.
- Does AppSec believe the role benefit from Secure Code Training anyway?

If a role should be added or removed, please open an MR and @ mention @gitlab-com/gl-security/product-security/appsec. If your role is in the list already but SCW doesn't show up in Okta, ask for help in secure-code-warrior on Slack. An AppSec team member will add you to the Google Group and update any baseline entitlements / onboarding checklists to ensure those in your role are included in the future.

The roles are:

- ceofullstackengineer
- communityrelationsfullstackengineer
- communityrelationsseniorfullstackengineer
- communityrelationsstafffullstackengineer
- corporateititsecurityanalyst
- corporateititsystemsengineer

- corporateitsenioritcomplianceengineer
- corporateitsenioritsecurityengineer
- corporateitsenioritsitereliabilityengineer
- corporateitsenioritsystemsengineer
- corporateitstaffitsystemsengineer
- customersupportmanagersupportengineering
- customersupportseniorsupportengineer
- customersupportstaffsupportengineer
- customersupportsupportengineer
- dataanalyticsengineer
- datadataengineer
- datadatascientist
- datauniordataengineer
- datasenioranalyticsengineer
- dataseniordataengineer
- dataseniordatascientist
- datastaffanalyticsengineer
- datastaffdataengineer
- datastaffdatascientist
- developmentassociatebackendengineer
- developmentassociatefullstackengineer
- developmentbackendengineer
- developmentdistinguishedengineer
- developmentdistributionengineer
- developmentengineeringfellow
- developmentfrontendengineer
- developmentfullstackengineer
- developmentmanagerengineering
- developmentmanagerfrontendengineering
- developmentmanagerfullstackengineering
- developmentprincipalengineer
- developmentprincipalfullstackengineer
- developmentseniorbackendengineer
- developmentseniordistinguishedengineer
- developmentseniordistributionengineer
- developmentseniorfrontendengineer
- developmentseniorfullstackengineer
- developmentseniorvulnerabilityresearchengineer
- developmentstaffbackendengineer
- developmentstafffrontendengineer
- developmentstafffullstackengineer
- developmentstaffvulnerabilityresearchengineer
- digitalexperiencefullstackengineer
- digitalexperiencemanagerengineering
- digitalexperiencemanagerengineeringmarketing
- digitalexperiencemarketingfrontendengineer
- digitalexperienceseniorfrontendengineer
- digitalexperienceseniormarketingfrontendengineer
- enterpriseapplicationsintegrationsengineer
- enterpriseapplicationsseniorintegrationsengineer

- incubationengineeringsseniorfullstackengineer
- incubationengineeringstaffincubationengineer
- infrastructurebackendengineer
- infrastructuredistinguishedengineerinfrastructure
- infrastructureengineeringfellowinfrastructure
- infrastructureseniorbackendengineer
- infrastructureseniordatabasereliabilityengineer
- infrastructureseniorsitereliabilityengineer
- infrastructuresitereliabilityengineer
- infrastructurestaffbackendengineer
- infrastructurestaffsitereliabilityengineer
- marketingopsseniordataengineer
- peopleoperationsfullstackengineer
- qualitybackendengineer
- qualitymanagerqualityengineering
- qualityprincipalengineer
- qualityseniorbackendengineer
- qualityseniorengineeringanalyst
- qualityseniorsoftwareengineerintest
- qualitysoftwareengineerintest
- qualitystaffbackendengineer
- qualitystaffengineeringanalyst
- qualitystaffsoftwareengineerintest
- securitydistinguishedengineersecurity
- securitymanagersecurityengineering
- securityprincipalsecurityengineer
- securitysecurityengineer
- securityseniorsecurityengineer
- securitystaffsecurityengineer
- uxseniorfullstackengineer

Teams to follow up with manually for additional enrollments:

- Security Assurance

</details>

Other Training

Language Specific Guidelines

- Secure Ruby on Rails with Jim Manico (Recommended)
- OWASP Secure Coding in Go (Recommended)
- Checkmarx Secure Coding in Javascript (Recommended)

Other Guidelines and Resources

- OWASP Top 10 2017 (Optional)
- OWASP API Security Top 10 (Optional)
- OWASP Serverless Top 10 (Optional)

- OWASP Mobile Top 10 (Optional)

Secure Coding Training with Jim Manico

Description

A developer-focused application security training presented by Jim Manico, and Dr. Justin Collins, the creator of Brakeman, occurred on the days of July 29th and 30th 2019. In addition to covering secure coding in general, it also covers specific threats and mitigations for Ruby on Rails applications. The content is presented in a lighthearted and entertaining manner.

You can find the recorded, private YouTube stream at the following:

- Day 1 Morning
- Day 1 Afternoon
- Day 2 Morning
- Day 2 Afternoon

These videos are private by default. To view them, you will need to switch to the GitLab Unfiltered account.

Recommendations

- Watch the video or read the slides for each topic that is relevant to you.
- The videos were recorded over two full days. It is suggested that you split up viewing them over multiple days by topic and/or by the hour.
- Consider watching the videos at 1.25X speed

Schedule and Topics

Day 1

Day 1 Morning

1. Introduction to Application Security (4:33)
1. Threat Modeling
1. OWASP Top Ten 2017 overview (42:57)
1. A1: Injection (52:03)
1. A2: Broken Authentication and Session Management (1:19:50)
1. A7: Cross site scripting - XSS (2:09:45)
1. A8: Insecure deserialization (2:15:10)
1. A9: Using known vulnerable components (2:22:26)
1. A10: Insufficient logging and monitoring (2:24:30)

Also covers:

- OWASP ASVS 4.0
- Multi-Form Workflow Security

Day 1 Afternoon

1. XSS Defense - HAML (1:51)
1. Safe client-side JSON Handling (1:45:31)
1. iFrame Sandboxing (1:57:25)
1. Input validation (2:04:50)
1. Unvalidated Redirects (2:22:14)
1. DevOps Best Practices (3:14:30)
1. Content Security Policy (3:36:31)
1. Brakeman and Static Analysis (4:09:20)

Also covers:

- DevOps Best Practices ·
- Coding Vue.js applications securely
- File Upload and File IO Security ·Multi-Step Secure File Upload Defense, File I/O Security
- Input Validation ·Basics ·(Allowlist Validation· and Safe Redirects)
- 3rd Party Library Security Management ·(·Detect and manage insecure 3rd party libraries)

Day 2

Day 2 Morning

1. Access control (4:28)
1. Insecure direct object reference in Rails (58:20)
1. Cross site request forgery (1:28:33)
1. Cross site request forgery protection in Rails (1:52:32)
1. Cookie Options and Security (2:33:45)
1. Server Side Request Forgery SSRF (2:44:22)

Also covers:

- Dynamic render paths and local file inclusion
- IDOR and scoped queries
- SSRF Defense
- Cross Site Request Forgery CSRF Defenses for multiple architecture types (stateless, API,web, etc)

Day 2 Afternoon

1. Authentication Best Practices (5:40)
1. Rails 6 Security Features (2:23:15)
1. Introduction to the OAuth authorization protocol v1 (2:48:21)
1. OAuth v2 (2:51:05)
1. Client Registration (3:04:06)
1. Authorization Code Grant (3:07:44)
1. OAuth 2.0 Terminology (3:21:06)
1. OAuth 2.0 Tokens (3:35:35)

Also covers:

- Secure Secret Storage

- Encrypted secrets/credentials

Recommended topics by role

Frontend Engineers

- Day 1
 - Intro to application security (4:33)
 - Threat modeling
 - OWASP top 10 overview (42:57)
 - Injection (52:03)
 - Broken authentication (1:19:50)
 - Cross site scripting (2:09:45)
 - Insecure deserialization (2:15:10)
 - Using vulnerable components (2:22:26)
 - XSS defense (1:51)
 - Safe client side JSON handling (1:45:31)
 - iFrame sandboxing (1:57:25)
 - Unvalidated redirects (2:22:14)
 - Content security policy (3:36:31)
 - Brakeman and static analysis (4:09:20)
- Day 2
 - Access control (4:28)
 - Cross site request forgery (1:28:33)
 - Cookie options and security (2:33:45)
 - SSRF (2:44:22)
 - Authentication (5:40)
 - Rails6 security features (2:23:15)
 - OAuth (2:51:05)

Backend Engineers

- Day 1
 - Intro to application security (4:33)
 - Threat modeling
 - OWASP top 10 overview (42:57)
 - Authentication and session management (1:19:50)
 - Insecure deserialization (2:15:10)
 - Vulnerable components (2:22:26)
 - Logging & monitoring (2:24:30)
 - XSS defense (1:51)
 - Input validation (2:04:50)
 - DevOps best practices (3:14:30)
 - Brakeman (4:09:20)
- Day 2
 - Access control (4:28)
 - Insecure direct object reference (58:20)
 - SSRF (2:44:22)
 - Authentication best practice (5:40)
 - Rails 6 security features (2:23:15)

- OAuth (2:51:05)

SRE

- Day 1
 - Intro to application security (4:33)
 - Threat modeling
 - OWASP top 10 overview (42:57)
 - DevOps best practices (3:14:30)

Additional resources

- PowerPoint presentations
- Questions Doc
- Ruby on Rails security best practices
- Online Labs
- Burp Proxy

SECTION: security/security-and-technology-policy-exception.md

Exceptions

Exceptions to Security and Technology Policies must be tracked and approved by the policy approver(s) via an auditable format. An exception process should be defined in each policy.

Information security considerations such as regulatory, compliance, confidentiality, integrity and availability requirements are most easily met when companies employ centrally supported or recommended industry standards. Whereas GitLab operates under the principle of least privilege, we understand that centrally supported or recommended industry technologies are not always feasible for a specific job function or company need. Exceptions from the aforementioned standard or recommended technologies is discouraged. However, it may be considered provided that there is a reasonable, justifiable business and/or research case for a Security and Technology Policy exception; resources are sufficient to properly implement and maintain the alternative technology; the process outlined in this and other related documents is followed and other policies and standards are upheld.

In the event a team member requires an exception from the standard course of business or otherwise allowed by policy, the requester must submit an Exception Request to the Security Assurance Department, which contains, at a minimum, the elements outlined in the issue template.

Exception request approval requirements are documented within the issue template. The requester should tag the appropriate individuals who are required to provide an approval per the approval matrix.

If the business wants to appeal an approval decision, such appeal will be sent to Legal at legal@gitlab.com. Legal will draft an opinion as to the proposed risks to the company if the exception were to be granted. Legal's opinion will be forwarded to the CEO and CFO for final

disposition.

Any exception approval must:

- Recommended compensating controls to reduce exposure and/or harm (i.e. admin rights to financially significant system may require audit logs and review of users activity within the system)
- Be captured in the associated exception request issue. All sections in the "Approvals" area of the issue must be filled in.

Once an exception request is submitted, the following general flow will commence:

- Requestor will receive approval from their Department head or the policy, standard, or procedure maintainers, depending on the type of exception request.
- Once approved by the requestor's Department, the team that owns the associated policy will review the exception request.
 - Security Compliance will determine whether the exception request has any current or planned compliance implications.
 - The Policy Owner will review the request to ensure appropriate compensating controls have been documented and will consider inputs from Security Compliance and other SMEs when determining the overall level of risk associated with the request.
- The Policy Owner will document a final decision, and if appropriate, a recommended action plan to mitigate risk from the exception request.
- The exception will be logged in the central exception management space.
- The exception will be reviewed as its expiration date approaches, and any extension of the exception will require a new and approved extension request.

SECTION: security/security-change-management-procedure.md

Purpose

The purpose of this document is to outline the procedural change management steps as they relate to the Security Division.

Scope

This document applies to systems and applications owned by Security and processes owned by Security Departments.

{{% note %}}

Changes related to user access and authorization should continue to be handled via the access request process.

{{% /note %}}

Security has defined the following types of changes:

Minor change

A minor change is a non-material change which occurs through the natural course of business, such as handbook updates. Minor changes may be implemented directly in Production, have no financial impact, are related to general maintenance, and can be easily reversed.

A minor change does not require a change management issue and can be handled internally within the relevant team. Security's least privilege access controls support the checks and balances required for managing minor changes.

Standard change

A standard change is a change that is low risk, relatively common and follows a specified procedure or work instruction, such as a configuration change, security patches, or other types of normal application modifications.

Standard changes have to go through the change management process. They require a peer review, Impacted Team(s) Management (or Code Owner) approval, and post-implementation review.

{{% note %}}

Manager - prior to approving the change request, please ensure that the correct change request template is being used.

{{% /note %}}

Comprehensive change

A comprehensive change is high risk, high impact, or has a more complex procedure, such as a system or application deprecation, a new system or application onboarding, or net-new Production architecture.

- Infrastructure changes are considered comprehensive changes. They require peer review, Impacted Team(s) Management (or Code Owner) approval, Technical Owner Approval, and Post-Implementation Review.

{{% note %}}

Manager - prior to approving the change request, please ensure that the correct change request template is being used.

{{% /note %}}

Emergency change

An emergency change follows the same approval process as comprehensive changes.

- It can be entered for approval after the change has been implemented in production.
- Emergency changes are intended to be used only where there is an immediate critical need to correct an operational or security issue that is preventing users from working or critical processes from operating.
- Emergency changes require review and all approvals after the change has been implemented.

Roles & Responsibilities

| Role | Responsibility |

| ----- | ----- |

| Security Governance | Maintain a security change management procedure to intake and respond to change management activities |

| | Maintain Metrics |

| Security Compliance | Provide oversight to ensure changes are being made in accordance with compliance obligations |

| Requestor | Complete the change management issue |

| | Work with the Technical Owner to document, test, and obtain approval(s) for the change |

| Technical Owner | As defined in the tech stack. Review and provide approval prior to the change being implemented |

| | Work with the Requestor to ensure the requested change is documented, tested, and approval(s) have been completed |

| | Ensure Peer Review is completed prior to providing Technical Owner Approval |

| | Document and report any risks or trends identified during change management activities |

| Peer Review | Review and ensure the requested change has been documented and there are no undocumented downstream impacts |

| Post Implementation Review | Review of the change in production after the change is made to ensure everything is working as expected |

Approval Matrix

Approval Type	Description	Minor	Standard	Comprehensive	Emergency
Peer Review	Peer Reviews are performed by a peer of the change Requestor and are intended to identify any potential issues with the planned change or change process. Note: The peer review process was established to mitigate the risk of the lack of segregation of duties between Requestor and implementer. The review provides comfort that changes to the production environment are valid.	No	Yes	Yes	Yes
Post-Implementation Review	Performed by a peer of the change Requestor and is intended to ensure the change is working as expected after the change has been implemented in Production.	No	Yes	Yes	Yes
Impacted Team(s) Management/Code Owner approval	Approval by Management that is responsible for the particular system or application	No	Yes	Yes	Yes
Technical Owner Approval	Approval by the system or application's Technical Owner as defined in the tech stack.	No	No	Yes	Yes

{{% note %}}

Technical Owner approval is dependent upon the system or application already existing in the tech stack. For new systems or applications that have not yet been assigned a Technical Owner in the tech stack, reach out to the Security Risk Team.

{{% /note %}}

Procedure

To submit a security change request, use one of the links below to create a change request issue:

[Standard Change Request](#)

[Comprehensive Change Request](#)

Emergency Change Request

CorpSec Change Request

Follow the instructions in the appropriate issue template to:

- Add the appropriate level of detail to the request
- Assign the appropriate team member(s) to the request in accordance with the roles and responsibilities and approval matrix sections

Security Change Management project

CorpSec Change Issues

Communication

If a security change request will impact all GitLab team members, please ensure that you communicate the change and its impact by posting in `whats-happening-at-gitlab` or `company-fyi-private` if the change is subject to SAFE or considered highly sensitive, before the work begins and after the change is completed.

Be sure to communicate the change, its rationale, and its impact.

If a security change request will impact all Security Division team members, please ensure that you communicate the change and its impact by posting in `security-team-only` before the work begins and after the change is completed.

Be sure to communicate the change, its rationale, and its impact.

If a security change request will impact a particular Security Department or individual team members, please ensure that you communicate the change and its impact in a manner appropriate for those impacted team members.

CorpSec - Rollout of Mass Deployments

Where possible, mass deployments such as macOS operating system upgrades (i.e. Monterey to Ventura) or net new software rollouts will be done by Divisions. Smaller changes such as security patches or minor updates of existing software are not subject to the same schedule and can be deployed globally after Phase 1 testing is complete. The order of mass deployments will be as follows:

Phase 1: Alpha Testers

Phase 2: Beta Testers

We welcome volunteers to join our Beta Testers group! Please join the `corpsec-beta-testers` channel on Slack. The list of Beta Testers is automatically updated daily via script using the list of Team Members in that channel. If you only want to join the `corpsec-beta-testers` channel but you do not want to be a part of the testing, please express your intention to opt-out of the testing at any time by stating as much in the channel and tagging a member of IT, or by

leaving the channel entirely.

Beta Testers are agreeing to participate in the testing of applications that may not be approved for country-wide implementation in certain jurisdictions. If you are in such a jurisdiction, your participation will be limited to Beta Testing and upon completion of the testing, access and use of the application will be revoked until GitLab receives approval for country-wide implementation of the application.

Phase 3: Security

Phase 4: Legal

Phase 5: Marketing, People Group, & CEO Division

Phase 6: Engineering & Product

Phase 7: Sales & Finance

Phase 8: VIPs

VIP support for significant or disruptive changes will be treated via direct support from the Service Desk team who will schedule a call at a convenient time to ensure the update or deployment is successful with minimal disruption.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

References

- Change Management Policy
- Controlled Documents Procedure
- Controlled Documents Annual Review Program

SECTION: security/security-culture.md

Mission Statement

The security department as a part of GitLab should follow and live up to the GitLab values and mission.

The transparency value can be especially difficult for a security department to embrace and embody, as due to the confidentiality of their work, security people tend to be secretive and intransparent by default.

Intent and goals

The intent of the security culture committee is to maintain a welcoming and transparent

environment within the security department.

The committee goals are to:

- Identify areas where our core values can be strengthened
- Improve transparency while maintaining security & privacy across the security organization
- Foster an inviting and welcoming environment for questions, concerns and feedback
- Propose ideas to promote teamwork and collaboration
- Help drive the mission of being the most transparent security group in the world
- Provide actionable feedback and direction to the security department, so they may best live up to the GitLab values

The committee should draft the ways to reach these goals for an open, approachable and transparent culture within the security department. The department's leadership should reinforce those ways by communicating and leading by example. The committee will provide an interface for all team members to express any concerns regarding the culture within the security department.

Participation

Current committee members:

- Madeline Lake
- Janina Roppelt
- Greg Myers
- James Hebden
- Ryan Lawson
- Clayton Shank
- Steve Ladgrove
- Ayoub Fandi
- Julie Davila (Executive Sponsor)
- Giuliana Lucchesi and Aurelia Akpan - (People Business Partner)

<details>

<summary>Previous committee members</summary>

Previously nominated committee members (July 2022 - September 2023):

- Jeff Burrows
- Mark Loveless
- Mitra Jozenazemian
- Rohit Shambhuni
- Will Szabo

Previously nominated committee members (January - June 2022):

- Chris Moberly
- Marie-Claire Cerny
- Nick Malcolm
- Valentine Mairet

Previously nominated committee members (FY22):

- Andrew Kelly
- Juliet Wanjohi
- Liz Coleman
- Philippe Lafoucrière

</details>

Participation

Participation in the committee is voluntary. The committee's goal is to have at least one team member from each sub-department within Security represented. In order to participate in the committee, please post in the security-culture slack channel and express your interest. Length of participation in the committee is at least 1 year in order to the committee to make a meaningful impact.

Meetings

The meetings will alternate between APAC & AMEA-friendly timezones. The recordings will be available in the GitLab Videos Recorded folder. Any team member is welcome to join - ask in security-culture if you can't find the event.

Process for change

To suggest a change, create an issue in the Security Culture Project.

The security culture committee has an issue template available for creating new issues.

Current Committee Projects

We are tracking our efforts in this issue: <https://gitlab.com/gitlab-com/gl-security/security-culture/-/issues/33>

Previous Security Committee Formation Documentation

The processes for forming the Security Culture Committee were updated in May 2024.

Previous documentation on forming the security culture committee and nomination processes can be found [here](#).

SECTION: security/security-division-dogfooding.md

Introduction and Purpose

The Security Division plays a critical role in ensuring the security of GitLab as a company, and of GitLab as a platform. In the pursuit of its charter, the division is committed to using ("dogfooding") GitLab as much as possible in order to help improve the platform for all

customers.

This dogfooding program aims to improve the effectiveness and efficiency of collaboration between Product Management and Security as it pertains to capabilities of the GitLab platform.

Historically, dogfooding within the Security Division has not been handled through a formalized program which has likely limited the success of dogfooding efforts.

Historical challenges include:

1. No framework for how feature requests and GitLab inventions and innovations should be prioritized when they come from the Security Division. This has led to some feature requests being indefinitely deferred, and the creation of multiple custom built tools which are used internally, representing functionality GitLab users could otherwise benefit from.
1. No strategic or executive level view for product leadership to understand current adoption, challenges, and desires from the Security Division.
1. Breakage of security processes as a result of bugs, poorly communicated breaking changes, and possibly other causal factors.
1. Lack of support and structure to help security team members who desire to contribute to the product to adapt their code to the quality and technology standards required to build their tooling as part of the product.
1. Limited contributions from Security Division members as a result of not having an assigned learning track for how to effectively contribute quality code to GitLab.

Desired Outcomes

These outcomes are all within the context of the Security Division. We will add links to the corresponding handbook pages as they are created.

1. Anyone at GitLab should be able to understand:
 1. How much of the GitLab platform is in use by the division.
 1. Which capabilities are used in their entirety, partially, or not at all by the division.
1. Product Management should be able to understand:
 1. Capabilities which are not used (in part, or in whole) because they don't meet operational requirements for the division
 1. Which feature requests are most important and critical to Security
 1. Which critical security activities rely on specific platform capabilities
 1. How to solicit feedback from any subset of the division with regard to new or updated capabilities
 1. What the division is planning to contribute to the product
1. Anyone within the Security Division should be able to understand:
 1. How to request a feature enhancement on behalf of the division and how to follow-up on that request.
 1. How to provide effective feedback to Product Management and Product Security Engineering.
 1. How to contribute quality code contributions back to the platform.
 1. Which custom tools are used by the division, and their rationale

Program Leadership and Stakeholders

Program DRI: TBD

Program Manager: TBD

Security Division Contacts

- VP, Product Security: Julie Davila
- Distinguished Security Architect: Philippe Lafoucrière
- Manager, Product Security Engineering: Andrew Kelly
- Principal Security Engineer: Dominic Couture

Product Management Contacts

TBD

Current Dogfooding Status

As of FY25Q2, we are evaluating our current utilization of GitLab features, enhancing communication between the Product and Security teams, and developing metrics to track the Security team's transition towards using GitLab products instead of custom tools, as well as our contributions to the product development.

Communication Cycles

1. Dogfood status reporting: Quarterly async updates
1. Critical PM escalations: ?
1. Requests for evaluations from PM to security

Key Processes

Management of Security Personas

GitLab maintains an official list of Personas describing different types of GitLab users, their needs, and the way they interact with GitLab products. The Security Division has representation through several of these personas. When a Security division team member requests a particular feature or functionality to be integrated into the product, an assessment is performed to determine if it would be appropriate to create a new Persona or modify an existing one.

Methodology for Dogfooding Assessment

Alignment with Product Stages and Information Hierarchy

The organization of the presentation layer of dogfooding data should be done in a way to maximize the efficacy of output. To this end, information will generally be presented in alignment with product stages in order to facilitate feedback loops that already mimic the mental models already prevalent across GitLab.

Unless pragmatism dictates otherwise, the output of dogfooding analysis should be organized by

product stage.

Documenting Capabilities Used, Partially Used, and Unused

Used

- 1. Current satisfaction
- 1. Desired improvements

Partials

- 1. Partial because of limitations
- 1. Partial because of internal capacity

Unused

- 1. Due to limitations
- 1. Lack of awareness
- 1. Planned, not yet being worked on

Management of Desired Product Capabilities

- 1. Feature requests
 - 1. Creation of user stories
 - 1. Establishing criticality of need
 - 1. Relevance to external customers
- 1. Considerations of build vs buy
 - 1. Wait for PM
 - 1. Delegate to product security engineering
 - 1. Hire contractors to build
 - 1. Purchase externally

Management of in-house tooling

Security division teams should strive to make contributions to the product instead of creating custom in-house tooling.

When the decision is made to create custom tooling outside of the GitLab product:

- 1. Document the reasoning and decision to build custom tooling in the README
 - 1. When applicable, document what the minimum set of GitLab features required to sunset the tool are
 - 1. Include link to issues and create new issues for those that are missing
 - 1. Otherwise document why the custom tool doesn't fit in GitLab's vision and why we are not trying to build it as part of the product
- 1. Identify and document clear ownership of what team will be responsible for maintaining the custom tooling

Progressive sunsetting of in-house tools

When new features are added to the product they will sometimes only partially meet our requirements. We should still attempt to remove custom code and logic as much as possible.

Some possible examples of this are:

- Removing the monitoring of commits from our secrets monitoring tooling when the Secrets Push Protection feature is fully available
- Replacing the custom categorization of projects using YAML files in the inventory and replacing it with Compliance Frameworks labels when we can assign multiple labels
- Use the Dependencies API instead of the inventory in gem-checker when the API becomes stable

In each situation the custom tool is still in use, but we dogfood more of the product and can give early feedback to the teams about what the issues are, if any.

Management of critical security practices reliant on the GitLab platform

1. Assessments of breaking changes
1. Escalations for critical flaws/bugs introduced

Evaluating and assessing updated or newly introduced platform capabilities

1. Discovery of new GitLab features
 1. Continuous communication with Product stakeholders
 1. Review of monthly release blog posts for minor and major versions to identify potential new beneficial features
 1. Compare new features with the current needs of the division identified in the Management of Desired Product Capabilities process
 1. Loop back new information into management of in-house tooling process

SECTION: security/security-shadow-sec-eng-res.md

Completion of each course you will receive a certificate. At the completion of all 3 courses your name will be recognized on this page.

Restrictions

Please keep in mind that there are some restrictions on what can and cannot be shared as part of the shadow program, particularly related to high severity vulnerabilities or incidents.

For example if a shadow is watching an AppSec team member triage HackerOne issues and a High or Critical vulnerability is reported, the shadow call should end.

Application Security

Do you like complexity? Do you have a knack for identifying insecure edge cases, communicating them and helping provide alternative solutions? Do you enjoy spending time in confidential issues? How about sifting through false positives, duplicates, spam, and other invalid security reports to improve the signal to noise ratio? If your response to each of these

questions is an enthusiastic "Yes," then there's a strong likelihood that the work of the AppSec Team will interest you.

Schedule / Topics Covered:

- AS101.1: Intro to AppSec (1 hour)
- AS101.2: HackerOne "The Program" (1-2 hours)
- AS101.3: AppSec Review Ride-Along (1-2 hours)
- AS101.4: Stable Counterparts and Working with Engineering Teams (1 hour)
- AS101.5: Security "It's Happening" Release (1-2 hours)

This schedule is a suggestion. The AppSec Engineer and the shadow are encouraged to communicate ahead of time or during the first session to understand the interests of the shadow to adapt the schedule in a way that provides as much value as possible to them.

It is also suggested to have a Google Doc to write down questions that might come between sessions.